Priority vs SRTF Comparison Project

DOCUMENTATION

1. Assumptions & Documented Rules:

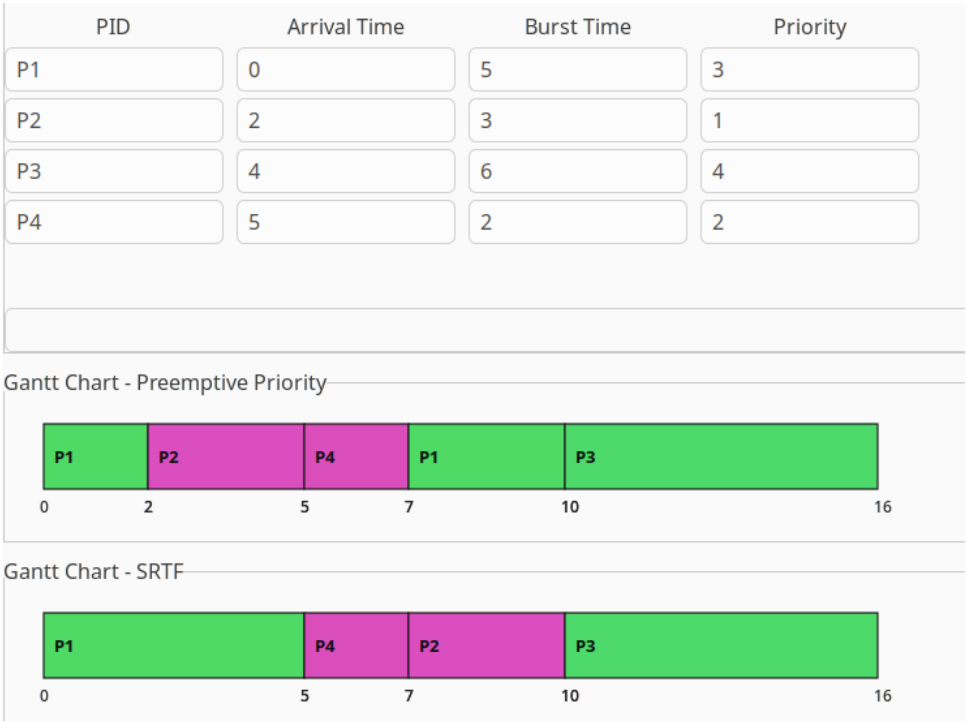
- **The priority rule:** Smaller integer values represent higher priority. For example, a process with Priority 1 will preempt a process with Priority 2.
- **The Tie Breaking rule:** If two processes share the same priority (in Priority Scheduling) or the same remaining burst time (in SRTF), the tie is resolved based on their Arrival Time.
- **Preemption:** Our Priority scheduling is assumed to be preemptive.

Important Rules:

- Turnaround Time (TAT) = Completion Time – Arrival Time
- Waiting Time (WT) = Turnaround Time – Burst Time
- Response Time (RT) = First Start Time – Arrival Time

2. Screenshots of Test Scenarios:

Scenario A (Normal Mixed Workload):



This scenario uses a normal mix of arrival times, burst times, and priorities. It also shows our tie-breaking rule.

1- Priority Behavior:

- P1 starts first because it's the only job at time 0.
- At time 2, P2 arrives. P2 has higher priority (1) than P1 (3). So, the system interrupts P1 and runs P2.
- At time 4, P3 arrives. But P2 has higher priority (1) than P3 (4). So, the system continues with P2.
- At time 5, P2 finishes and P4 arrives. The system compares priorities between P1, P3 and P4. P4 has higher priority and runs it for 2 seconds.
- After P4 finishes, the system runs P1 then P3 because P1 has higher Priority.

2- SRTF Behavior:

- P1 starts first because it's the only job at time 0.
- At time 2, P2 arrives. But the burst time for P1 at that moment is 3 and P2 is 3. So, the system continues running with the earliest arrival time as a tiebreaker for P1 and finishes it.
- At time 5, P1 is finished. Now the system looks for waiting process: P2 has 3 burst time, P3 has 6 burst time, P4 has 2 burst time.
- P4 has the shortest burst time. So, the system runs P4 first then P2 and at last P3.

Priority Result Table:

PID	TAT	WT	RT
P1	10	5	0
P2	3	0	0
P3	12	6	6
P4	2	0	0
Average	**6.75**	**2.75**	**1.5**

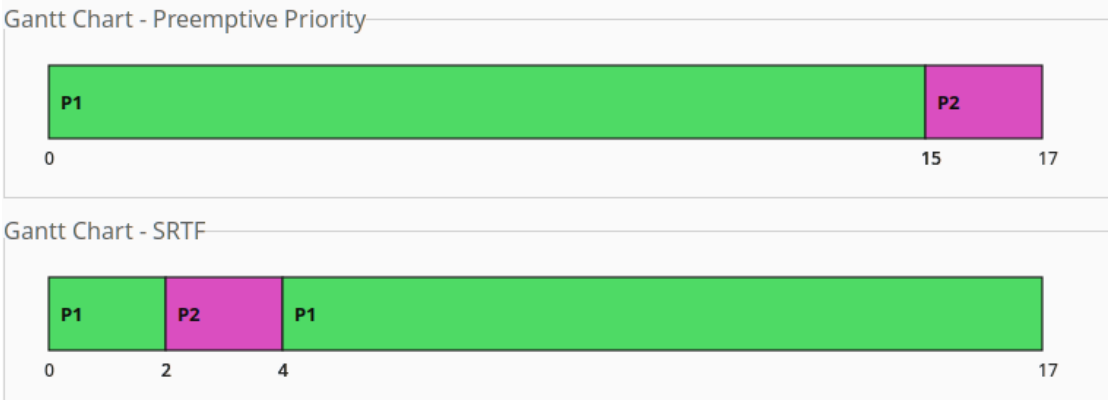
SRTF Result Table:

PID	TAT	WT	RT
P1	5	0	0
P2	8	5	5
P3	12	6	6
P4	2	0	0
Average	**6.75**	**2.75**	**2.75**

Conclusion: While TAT and WT were equal, RT in Priority has faster RT than in SRTF (1.5 vs 2.75). So, this workload priority provides better overall average performance.

Scenario B (Conflict between Priority and Burst Time):

PID	Arrival Time	Burst Time	Priority
P1	0	15	1
P2	2	2	4



This scenario includes a high-priority long process (P1) and a low-priority short process (P2). It clearly demonstrates the main difference between Priority and SRTF.

1- Priority Behavior:

- P1 starts first because it's the only job at time 0.
- At time 2, P2 arrives. P2 is a very short job, but it has lower priority than P1.
- The system continues running P1 and after finishing runs P2.

2- SRTF Behavior:

- P1 starts first because it's the only job at time 0.
- At time 2, P2 arrives. The system compares the burst time for each. P1 has 13 units left, but P2 has 2 units left.
- P2 preempts P1. P2 runs for 2 units until finishing at time 4, then the system allows to run P1.

Priority Result Table:

PID	TAT	WT	RT
P1	15	0	0
P2	15	13	13
Average	**15**	**6.5**	**6.5**

SRTF Result Table:

PID	TAT	WT	RT
P1	17	2	0
P2	2	0	0
Average	**9.5**	**1**	**0**

Conclusion: For this workload, SRTF provides better overall average performance. However, Priority has successfully protected the urgent high priority task P1 from being interrupted.

Scenario C (Starvation-sensitive case):

PID	Arrival Time	Burst Time	Priority
P1	0	2	4
P2	1	8	1
P3	2	8	1
P4	3	8	1

Run Simulation

This scenario demonstrates the concept of "starvation." We set up one low-priority job (P1) that gets constantly interrupted by high-priority jobs (P2, P3, P4).

1- Priority Behavior:

- P1 starts first because it's the only job at time 0.
- At time 1, P2 arrives. P2 has higher priority (1) than P1 (4). So, the system interrupts P1 and executes P2.
- P3 and P4 arrive while P2 is still running, and they have the same priority as P2. So, the system looks to the arrival time. P2 runs until finish then P3 then P4.
- Starvation Occurs: P1 is forced to wait a huge amount of time just to execute its 1 unit remaining.

2- SRTF Behavior:

- P1 starts first because it's the only job at time 0.
- At time 1, P2 arrives. But it has a higher burst time than P1. So, the system finishes P1.
- At time 3, P3 and P4 arrive, with the same burst time as P2. System executes them with arrival time, P2 first then P3 then P4.

Priority Result Table:

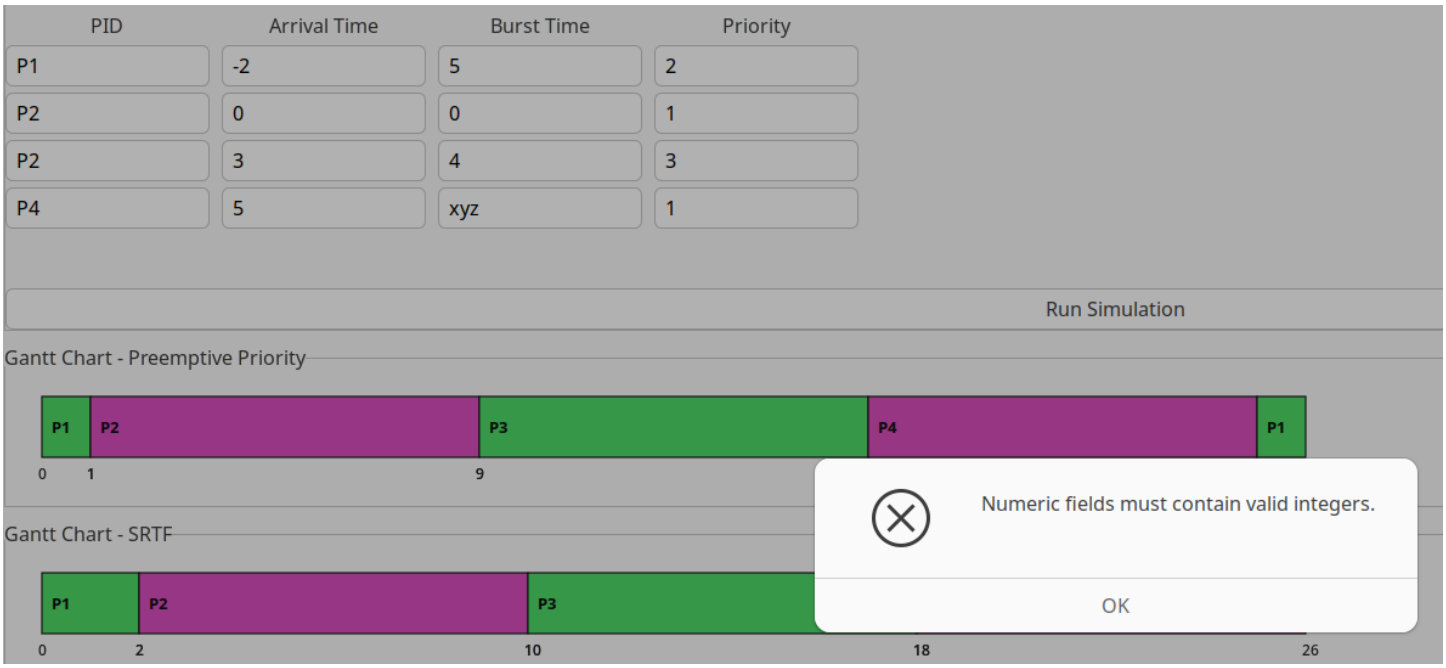
PID	TAT	WT	RT
P1	26	24	0
P2	8	0	0
P3	15	7	7
P4	22	14	14
Average	**17.75**	**11.25**	**5.25**

SRTF Result Table:

PID	TAT	WT	RT
P1	2	0	0
P2	9	1	1
P3	16	8	8
P4	23	15	15
Average	**12.5**	**6**	**6**

Conclusion: This workload highlights the starvation risk in Priority scheduling. Because of the constant arrival of high-priority tasks, P1 suffered a massive Waiting Time of 24. SRTF handled this workload much better in terms of average TAT and WT, as it allowed the nearly finished short job to clear out of the system immediately.

Scenario D (Validation Case):



This scenario demonstrates error handlings and input validation.

Invalid Input Tested:

- **Negative Arrival Time:** P1 has Arrival Time of –2.
- **Zero Burst Time:** P2 has Burst Time of 0.
- **Duplicated Process IDs:** P2 is entered twice.
- **Non-Numeric Input:** P4 has Burst Time of xyz.

3. Analysis Questions:

1. **Which algorithm produced the lower average waiting time?**
   - In Scenarios B and C, SRTF produced lower average waiting time.

2. **Which algorithm produced the lower average response time?**
   - The results were mixed. SRTF generally provided lower response times for shorter jobs, but Priority Scheduling could force lower response times for specific jobs if they were assigned a high priority on arrival (Scenario A).

3. **Did priority values improve treatment of urgent processes?**
   - Yes. In Scenario B, the priority values protected the urgent, high-priority long process (P1) from being interrupted by a shorter, low-priority process.

4. **Did SRTF favor short jobs more aggressively?**
   - Yes. In both Scenarios B and C, SRTF aggressively favored the shortest remaining jobs, preempting longer jobs immediately to clear the short tasks out of the system.

5. **Which algorithm would you recommend for the tested workload, and why?**
   - I would recommend SRTF in general systems. Because it provides the best performance and minimizes the average waiting time.
     However, in specialized systems, I would recommend Priority Scheduling to ensure important tasks are finished first and not delayed by shorter ones.

4. Conclusion:

1. **Performance:** Overall, SRTF performed better across Scenario B and C, while Scenario A was relatively tied.

2. **Metrics:** SRTF provided better Turnaround Time (TAT) and Waiting Time (WT), While Priority Scheduling provided better Response Time (RT) for specific high-priority tasks.

3. **The Main Trade-Off:** It was **Efficiency** vs. **Policy**. SRTF is highly efficient. Priority Scheduling enforces strict urgency rules but sacrifices overall system efficiency to do so.

4. **Fairness in Practice:** SRTF appeared fairer in practice. While it strongly favors short jobs, all jobs eventually progress. Priority Scheduling was noticeably unfair to low-priority tasks, resulting in starvation (Scenario C) where a nearly finished task was forced to wait for an extreme amount of time.